

Domande Orale ASD (Camurati)

- **Divide et Impera + Decrease and conquer:**

- Divide et Impera è un paradigma di progettazione algoritmica che:
 1. Divide il problema in sottoproblemi più piccoli
 2. Risolve ricorsivamente i sottoproblemi
 3. Combina le soluzioni per ottenere quella globale
 - Es. : Merge e Quick Sort, Bynary Search, Karatsuba
- Decrease and Conquer -> si risolve un problema
 - Riduco a UN SOLO sottoproblema più piccolo (n-1,n-2,...)
 - Si costruisce la soluzione finale usando quella del sottoproblema
 - Tipi:
 - By one -> riduco dim di 1
 - By constant factor -> divido dim per un fattore k
- Divide -> PIU' SOTTOPROBLEMI PIU' PICCOLI
- Decrease -> UN SOLO SOTTOPROBLEMA RIDOTTO

- **Merge Sort: complessità, eq. alle ricorrenze, stabilità e se è in loco**

Il Merge Sort è un algoritmo ricorsivo di tipo **Divide et Impera**.

È stabile perchè durante il merge, se due elementi sono uguali, viene preso per primo quello che sta prima.

Questo accade perchè si usa il confronto $\leq$ nel merge.

Complessità:

- Tempo: $O(n \log n)$ in tutti i casi (Migliore, medio e peggiore)
- Spazio: $O(n)$ per l'array temporaneo usato nel merge.

Equazione delle ricorrenze:

- $2T(n/2)$: divido in due sottoproblemi
- cn : tempo per il merge.

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

- Divide in due metà, ordina ricorsivamente, poi merge

- **Complessita' caso peggiore del Quicksort, in che situazione capita la complessita' di caso peggiore?**

- Il Quick Sort è un algoritmo di ordinamento ricorsivo basato sul paradigma Divide et Impera. Funziona così:
 1. Sceglie un pivot (un elemento dell'array)
 2. Partiziona l'array in due sottosequenze:

- A sinistra del pivot: tutti gli elementi minori o uguali
 - A destra del pivot: tutti gli elementi maggiori
- 3. Applica ricorsivamente lo stesso procedimento alle due sottosequenze
- Caso medio:
 - Il caso medio ha complessità $O(n \log n)$. Succede quando il pivot divide l'array in parti abbastanza bilanciate.
- Caso peggiore:
 - Il caso peggiore è $O(n^2)$. Succede quando il pivot è sempre il minimo o il massimo. E.g.: array già ordinato o ordinato inversamente. In questo caso, il partizionamento genera una parte vuota e una da $n-1$, per ogni chiamata ricorsiva.
- Come evitarlo:
 - Per evitare il caso peggiore, prima di chiamare il quick sort. si può fare una permutazione casuale degli elementi. Il pivot ha probabilità molto bassa di essere sempre il minimo o il massimo
 - Si garantisce il caso medio con alta probabilità
- **Programmazione dinamica**
 - La programmazione dinamica (DP) è una tecnica di progettazione algoritmica che:
 - Risolve problemi complessi scomponendoli in sottoproblemi sovrapposti
 - Salva le soluzioni intermedie per evitare di ricalcolarle
 - È usata quando la ricorsione pura è inefficiente perchè ripete gli stessi calcoli molte volte.
 - Quando si può usare?
 - Sottostruttura ottima: la soluzione del problema si basa sulle soluzioni ottimali dei sottoproblemi.
 - Sovrapposizione dei sottoproblemi: gli stessi sottoproblemi vengono risolti più volte.
 - Approcci principali:
 - TOP-DOWN: Con memoization, ricorsivo, salva i risultati in una tabella quando vengono calcolati.
 - BOTTOM-UP: Con tabulazione, iterativo, calcola e memorizza le soluzioni in una tabella partendo dai casi base.
- **Cos'è un BST? Definisci altezza di un albero**
 - Un BST è un albero binario in cui per ogni nodo vale:

- Tutte le chiavi del sottoalbero sinistro sono minori della chiave del nodo.
 - Tutte le chiavi del sottoalbero destro sono maggiori della chiave del nodo.
- Ricerca, inserimento e cancellazione in tempo medio $O(\log n)$ se l'albero è bilanciato.
- **Rango di un BST e BSTpartition:**
 - Rango r:
 - Posizione r-esima nell'ordinamento delle chiavi, es. rango 0 -> item con chiave minima
 - Partition : riorganizza BST avendo item di rango k (k-esima più piccola) in radice
 - Considerando t -> numero nodi nel sottoalbero sinistro
 - $t > k$: ricorro sotto-sx, partiziono rispetto k-esima, al termine rotDX
 - $t < k$: ricorro sotto-dx, partiziono rispetto (k-t-1)-esima, al termine rotSX
- **Partizioni ed Algoritmo di Er**
 - Una partizione di un insieme è una scomposizione dell'insieme in sottoinsiemi disgiunti e non vuoti tali che:
 - Ogni elemento appartiene a uno e uno solo dei sottoinsiemi.
 - L'unione di tutti i sottoinsiemi è l'intero insieme originale
 - Er:
 - Algoritmo ricorsivo per generare tutte le partizioni di un insieme:
 - Parte da un insieme vuoto di partizioni
 - Ad ogni passo prende un nuovo elemento e prova a:
 - Inserirlo in ogni sottoinsieme esistente
 - oppure creare un nuovo sottoinsieme con solo quell'elemento.
 - Prosegue ricorsivamente fino a generare tutte le combinazioni valide
 - Esplora tutte le configurazioni possibili in modo sistematico. Genera tutte le partizioni dell'insieme e ha complessità esponenziale.
 - Non deriva da calcolo combinatorio
- **Cosa è una coda a priorità e come posso implementarla + complessità PQinsert**
 - Una coda a priorità è una struttura dati in cui ogni elemento ha una priorità (gestisce elementi in base alla loro priorità, non in ordine di inserimento).
 - Implementazioni:

- Vettore non ordinato: utile se hai poche extract ma tante insert
 - Insert: $O(1)$, Extract: $O(n)$
- Vettore ordinato decrescente: utile se hai poche insert ma tante extract
 - Insert: $O(n)$, Extract: $O(1)$
- Heap binario: Struttura con albero binario completo, salvato in un array
 - Insert: $O(\log n)$, Extract: $O(\log n)$
- Heap con vettore di posizione ($pq[]$, $qp[]$): Serve quando devo modificare la priorità di un elemento già presente.
 - Ottimizza PQchange in $O(\log n)$
- Fibonacci heap / Pairing Heap: utilizzati in algoritmi avanzati (Tipo Dijkstra ottimizzato):
 - Insert $O(1)$, extractMin $O(\log n)$
- **Complessità PQchange:**
 - PQchange modifica la priorità dell'elemento con indice i nella coda a priorità, aggiornandola a un nuovo valore p e poi ripristina le proprietà dell'heap:
 - PQchange con indice diretto del valore:
 - Sai dove si trova l'elemento (indice i nell'array $pq[]$)
 - Cambi la priorità e risali o riscendi per ripristinare l'heap
 - Funziona bene con heap e vettori di posizioni
 - $O(\log n)$
 - PQchange con chiave (non con indice):
 - Funziona bene con heap e vettori di posizioni
 - Devi cercarlo in tutta la coda $\rightarrow$ costo $O(n)$ in un array
 - $O(n)$
 - Abbattere complessità con $qp[]$:
 - Vettore ausiliario che ti dice in quale posizione dell'array $pq[]$ si trova un certo elemento (di chiave k o di indice i)
 - Funziona come una mappa inversa tra indice logico e indice fisico.
 - Ora troviamo elemento con $O(1)$
- **Algoritmo di Kruskal**
 - Algoritmo per trovare MST
 - Usa corollario per trovare arco sicuro
 - Costruisce la MST aggiungendo archi, partendo da un insieme vuoto:
 - Ordina tutti gli archi per peso crescente

- Aggiungi l'arco più leggero che non forma un ciclo con quelli già scelti
 - Continua finchè hai $V-1$ archi nella MST
- Complessità $O(E \log E)$
- **Algoritmo di Prim**
 - Algoritmo per trovare MST
 - Soluzione brute-force
 - Uso teorema dell'arco sicuro
 - Costruisce la MST aggiungendo archi, uno alla volta, partendo da un nodo:
 - Scegli un nodo iniziale
 - Iterazione di $V-1$ passi, aggiungo un arco alla soluzione
 - Iterazione sugli archi per selezionare:
 - Ad ogni passo, aggiungi l'arco di peso minimo che collega un nodo già inclusi a un nodo ancora fuori dalla MST, quindi che **ATTRAVERSA IL TAGLIO**
 - Ripeti finchè tutti i nodi sono inclusi
 - Strutture dati:
 - Matrice delle adiacenze
 - $St[]$: ha $G \rightarrow V$ el, registra i padri di ogni vertice
 - $Frangia []$: $G \rightarrow V$ el, per ogni vertice di quelli non collegati, qual è il vertice più vicino che appartiene già alla soluzione
 - $Wt[]$: $G \rightarrow V+1$ el
 - per vertici di S: peso arco del padre
 - per vertici $V-S$: peso arco verso vertice di S più vicino
 - inizializzato con $\max Wt$
 - var min: vertice di $V-S$ più vicino ai vertici di S
 - Complessità:
 - Per grafi densi: $O(|V|^2)$
 - Per grafi sparsi
 - PQ implementata con heap: $O(|E| \log |V|)$
- **ADT union find**
 - Struttura dati per memorizzare una collezione di insiemi disgiunti
 - Operazioni di UFind (fusione 2 sottoinsiemi) e UFind (verifica se 2 elementi sono nello stesso sottoinsieme)
 - Dati N elementi, da 0 a $N-1$:
 - $id[]$: per ogni el, indice di chi lo rappresenta, all'inizio se stesso
 - $sz[]$: cardinalità del sottoinsieme a cui appartiene l'elemento
 - Weighted Quick-Union:

- Quick-union: elemento punta a chi lo rappresenta, facile
 - $O(1)$
 - Find: percorro catena fino al rappresentante ultimo del sottoinsieme, fondo il sottoinsi più piccolo in quello maggiore
 - $O(\log n)$
- UF rappresenta SOLO insiemi DISGIUNTI
 - Non permette di effettuare operazioni insiemistiche generiche come intersezione, differenza.
- Componenti connesse e fortemente connesse e differenza tra le due
 - Una **componente connessa** è, in un grafo non orientato, un sottoinsieme di vertici in cui ogni coppia è collegata da un cammino.
 - Ogni albero della DFS è una CC
 - Una **componente fortemente connessa**:
 - ALGORITMO DI KOSARAJU:
 - Grafo trasposto G'
 - DFS di G' :
 - Ordino vertici per tempo di fine elaborazione decrescenti
 - DFS di G seguendo ordine dei vertici come sopra
 - Ogni albero di questa DFS è una SCC
 - Differenza: la “fortemente connessa” vale solo per grafi orientati e richiede la raggiungibilità in entrambe le direzioni.
- Kernel DAG + perché è aciclico
 - Considero un vertice di ogni SCC come rappresentante della SCC
 - Estratto un grafo ridotto G'
 - E' un DAG, detto Kernel DAG
 - Se non lo fosse non connetterebbe SCC
- Tipi di archi in un grafo orientato(T, B, F, C)
 - Es: v (destinazione) w (sorgente) arco ($w-v$):
 - Arco Tree: il tempo di scoperta di v ($pre[v]$) deve essere maggiore del tempo di scoperta di w ($pre[w]$).
 - Arco Back: il tempo di fine elaborazione di v ($post[v]$) deve essere maggiore del tempo di elaborazione del vertice di partenza ($post[w]$)
 - Arco Forward: il tempo di scoperta di v deve essere maggiore del tempo scoperta di w
 - Arco Cross: il tempo di scoperta di v deve essere maggiore del tempo di scoperta di w .
 - Nel grafo non orientato troviamo solo archi T e B
- Heap e domande riguardo la struttura e i vantaggi e gli svantaggi:

- Heap è un albero binario:
 - Completo a sinistra (tutti livelli completi, tranne a volte ultimo, riempito da SX a DX) -> bilanciato
 - Key del padre $\geq$ chiave di ogni figlio
- Implementato come vettore
 - Accedo a max/min in $O(1)$
 - Inserisco o estraggo in $O(\log n)$
- Heapify:
 - Funzione che ripristina la proprietà dello heap a partire da un nodo, assumendo che i suoi figli siano già heap.
 - Ha un costo di $O(\log n)$
 - Come funziona:
 1. Confronta il nodo con i suoi figli
 2. Se uno dei figli viola la proprietà dell'heap, lo scambia con il più grande
 3. Ripete ricorsivamente la stessa cosa sul figlio scambiato
 - Condizioni
 - Nodo i preso in considerazione ha almeno 1 figlio (non foglia)
 - I sottoalberi sinistro e destro di i sono già degli heap
- BuildHeap:
 - Costruisce un heap da un array non ordinato applicando Heapify dal basso verso l'alto
 - A partire dal primo nodo non foglia, chiama Heapify(i) su tutti i nodi interni dal basso verso l'alto
 - Primo nodo non foglia è all'indice $(n/2) - 1$
 - Anche se Heapify ha costo $O(\log n)$ nel caso peggiore, somma dei costi -> serie pesata che converge a $O(n)$:
 - nodi in basso sono molti ma altezza piccola -> poco costo lì
 - nodi in alto pochi ma possono richiedere più lavoro
- Come si può implementare un grafo e vantaggi e svantaggi:
 - Matrice di Adiacenza:
 - $V \times V$ elementi
 - $Adj[i,j] = wt$ se esiste, 0 se non connessi i vertici
 - $wt = 1$ se grafo non pesato
 - Adj simmetrica per grafi non orientati
 - $O(V^2)$
 - Vantaggiosa solo per grafi densi
 - Non costi aggiuntivi per grafi pesati, cambio solo 1 in wt

- Accesso efficiente con $O(1)$
- Lista di Adiacenza
 - Vettore A di V elementi
 - $A[i]$ contiene il puntatore alla lista dei V adiacenti ad i
 - Vantaggi:
 - Grafi non orientati $\rightarrow 2 |E|$ el complessivi nelle liste
 - Grafi orientati $\rightarrow |E|$ el complessivi nelle liste
 - Svantaggi:
 - Scansione lista per verifica di adiacenza di 2 vertici
 - Uso di ulteriore memoria per pesi nei grafi pesati
 - $O(|V| + |E|)$
 - Vantaggioso per grafi sparsi
- Differenze e somiglianze strutturali tra BST e heap, come varia l'altezza
 - BST:
 - Ordinamento globale $\rightarrow$ figli SX tutti $\leq$ del padre e a DX $\geq$
 - Altezza dipende da ordine inserimento
 - Varia da $O(\log n)$ se bilanciato a $O(n)$ se degenerare a lista
 - Heap:
 - Sempre albero binario quasi completo
 - Altezza sempre $O(\log n)$
- Definizione di albero ricoprente minimo
 - Un sottoinsieme degli archi di un grafo non orientato, connesso e pesato
 - Estraggo da G un MST
 - Collega tutti i nodi senza cicli e con peso minimo
 - MST è unico $\leftrightarrow$ tutti i pesi sono distinti
 - In un MST di V vertici, ci sono sempre V-1 archi
 - Approccio completo:
 - Raggruppo i V-1 archi dagli E archi del grafo originale
 - Combinazioni $\rightarrow$ ordine non conta
 - Verifica di copertura ed aciclicità
 - Considero tutte le soluzioni e si sceglie la migliore
 - Costo esponenziale
 - Approccio Greedy:
 - Scelgo soluzione localmente ottima ad ogni passo
 - Non garantisce soluzione globalmente ottima
 - Per gli MST la soluzione risulta globalmente ottima
 - La soluzione corrente A è sottoinsieme degli archi di un MST
 - Inizialmente A insieme vuoto

- Ad ogni passo aggiungo ad A un ARCO SICURO
 - Fino a quando A non diventa MST
 - Arco è sicuro $\leftrightarrow$ aggiunto ad un sottoinsieme di MST produce ancora un sottoinsieme di MST
- **Dag e ordinamento topologico**
 - DAG $\rightarrow$ grafo aciclico orientato, non ha archi B
 - Non esistono percorsi che riportano ad un nodo già visitato
 - 2 particolari classi di nodi:
 - Nodi Sorgente: in-degree = 0
 - Nodi Pozzo: out-degree = 0
 - Ordinamento topologico $\rightarrow$ riordino i vertici secondo linea orizzontale
 - Se esiste arco (u,v) $\rightarrow$ vertice u a SX di v
 - Archi vanno tutti da SX a DX
 - Ogni DAG ha sempre almeno un ordinamento topologico
 - **Relazione tra cammino di Hamilton e DAG**
 - Se esiste un cammino hamiltoniano orientato
 - Ordinamento topologico è UNICO
 - Se non esiste
 - Non è unico
- **Cammini massimi (NP e su DAG)**
 - Problema non trattabile su grafi pesati qualsiasi
 - Assenza di cicli del DAG rende facile il problema:
 - Ordinamento topologico del DAG
 - Andando in ordine con i vertici dall'ordinamento:
 - Applico relaxation "Invertita" da quel vertice
 - Quindi aggiornare se AUMENTA la stima
- **Relaxation**
 - Si applica agli archi, consideriamo s come vertice di partenza
 - $d[v]$: stima del peso del cammino minimo da s a v
 - inizialmente
 - $d[v] = \max W_T, st[v] = -1$
 - $d[s] = 0, st[s] = 0$
 - relax:
 - se conviene il cammino tra s ed un intermedio u + peso arco (u,v)
 - allora $d[v] = d[u] + wt(u,v)$ e $st[v] = u$
 - Dopo la relaxation $d[v]$:
 - o è INVARIANTE: relaxation non ha avuto effetto
 - o è DIMINUITO: effetto del rilassamento

- **Algoritmo di Dijkstra**

- Archi con peso negativo?
 - Se non ci sono cicli $< 0 \rightarrow$ soluzione ottima non garantita
 - Se ci sono cicli $< 0 \rightarrow$ risultato senza significato
- Si applica una strategia GREEDY
- Strutture:
 - $S \rightarrow$ vertici per cui ho già calcolato peso per cammino minimo da s
 - $V-S \rightarrow$ coda a priorità PQ dei vertici ancora da stimare
- Termina quando PQ vuota:
 - Estrae vertice u da V-S, ovvero d[u] minimo
 - Inserisce u in S
 - Rilassa tutti gli archi uscenti da u
- Complessità:
 - Da sommare:
 - PQ vertici da stimare $\rightarrow O(V)$
 - Estraggo u da V-S $\rightarrow O(\log V)$
 - Inserisce u in S $\rightarrow O(\log V)$
 - Rilassa tutti gli archi uscenti da u $\rightarrow O(E)$
 - $O((V + E) \log V)$
 - $O(E \log V)$
- Per DAG pesati
 - Stessa cosa dei cammini massimi
 - Ordine vertici dall'ordinamento topologico
 - Complessità
 - $O(V + E)$

- **Bellman-Ford**

- Archi con peso negativo?
 - Se non ci sono cicli $< 0 \rightarrow$ soluzione ottima garantita
 - Se ci sono cicli $< 0 \rightarrow$ rileva il ciclo a peso negativo, me ne accorgo
- Usa PROGRAMMAZIONE DINAMICA
- Strutture:
 - $Inc(v) \rightarrow$ insieme dei vertici da cui escono archi incidenti in v
 - $d_w[v] \rightarrow$ lunghezza cammino minimo da s a v, al più w archi
 - $d_0[v] = 0$, se v coincide con s
 - $d_0[v] = \infty$, altrimenti
 - $d_w[v] =$ minimo tra
 - $d_{w-1}[v]$
 - $d_{w-1}[u] + wt(u,v)$

- Calcolo bottom-up:
 - Inizializzo per bene $d[]$ e $st[]$
 - Ordino archi in maniera lessicografica
 - $V - 1$ passi:
 - Ciclo di rilassamento sugli archi “in avanti” : lista di adiacenza
 - Al V -esimo passo:
 - Se diminuisce almeno una stima $\rightarrow$ presente ciclo < 0
 - Se non variano le stime $\rightarrow$ soluzione ottima trovata
 - La terminazione può avvenire anche prima
 - Se non cambiano stime prima del V -esimo passo
- Complessità
 - Devo sommare
 - Inizializzazione $\rightarrow O(V)$
 - $V-1$ passi, in ciascuno relaxo ogni arco $\rightarrow O(V \times E)$
 - V -esimo passo di controllo $\rightarrow O(E)$
 - $O(V \times E)$
- Teorema e corollario archi sicuri
 - Teorema:
 - Dato:
 - un grafo non orientato, pesato e connesso
 - $A \rightarrow$ sottoinsieme degli archi
 - Inizialmente vuoto
 - SE:
 - A è contenuto in qualche MST di G
 - $(S, V-S)$ è un taglio qualunque che rispetta A
 - (u, v) è un arco leggero che attraversa il taglio $(S, V-S)$
 - ALLORA:
 - (u, v) è un ARCO SICURO per A
 - Corollario:
 - Dato:
 - un grafo non orientato, pesato e connesso
 - $A \rightarrow$ sottoinsieme degli archi
 - Inizialmente vuoto
 - SE:
 - A è contenuto in qualche MST di G
 - C è un albero della foresta $G_A = (V, A)$

- (u,v) è un arco leggero che connette C ad un altro albero in G_A
 - ALLORA:
 - (u,v) è un ARCO SICURO per A
- **Trovare il successore e il predecessore in un BST e la complessità**
 - Successore:
 - Se esiste, è il nodo con Item con la più piccola key $>$ della chiave di Item del nodo h
 - 2 casi:
 - Se esiste sottoDX di h $\rightarrow \text{Succ}(h) = \text{minimo del sottoDX di h}$
 - Se non esiste sottoDX di h $\rightarrow \text{Succ}(h)$ è il PRIMO ANTENATO DI H, IL CUI FIGLIO SX E' ANCHE ANTENATO DI H
 - Predecessore:
 - Se esiste, è il nodo con Item con la più grande key $<$ della chiave di Item del nodo h
 - 2 casi:
 - Se esiste sottoSX di h $\rightarrow \text{Succ}(h) = \text{massimo del sottoSX di h}$
 - Se non esiste sottoSX di h $\rightarrow \text{Succ}(h)$ è il PRIMO ANTENATO DI H, IL CUI FIGLIO DX E' ANCHE ANTENATO DI H
 - Complessità:
 - E' una ricerca in un BST:
 - $O(\log N)$ se albero bilanciato
 - $O(N)$ se albero degenerare $\rightarrow$ una lista
 - Visita INorder, il predecessore/successore è l'elemento prima/dopo
- **Come funziona la BSTdelete?**
 - Per cancellare nodo con item con chiave k, bisogna mantenere:
 - Proprietà del BST
 - Struttura dell'albero binario
 - Passi:
 - Controllo se nodo da cancellare è in sottoSX o sottoDX del nodo corrente $\rightarrow$ se sì, ricorro nel sottoalbero
 - Se è la radice, elimino e ricombino i 2 sottoalberi
 - NUOVA RADICE $\rightarrow$ SUCCESSORE O PREDECESSORE DEL NODO CANCELLATO
 - Ovviamente bisogna ruotare
- **Complessità della rotazione nei BST?**
 - La complessità di una semplice rotazione è $O(1)$

- Se invece consideriamo una BSTpartition in cui bisogna partizionare un BST (di solito si effettua rispetto alla chiave mediana):
 - Consideriamo h come altezza del BST:
 - Se BST bilanciato $\rightarrow O(h) = O(\log n)$
 - Se BST degenerare, sbilanciato $\rightarrow O(h) = O(n)$
 - La complessità è direttamente proporzionale all'altezza del BST
- **Perchè conviene l'implementazione come vettore piuttosto che come liste?**
 - Attraverso un heap implementato come vettore posso individuare padre e figli di ogni nodo tramite semplici operazioni matematiche
 - Se considero indice i per un nodo:
 - $2i + 1 \rightarrow$ figlio SX
 - $2i + 2 \rightarrow$ figlio DX
 - $(i-1)/2 \rightarrow$ padre
- **Definizione di grafo connesso**
 - Un grafo non orientato si dice **connesso** se per ogni coppia di vertici esiste almeno un cammino che li collega
 - Non ci sono vertici "isolati" o gruppi separati $\rightarrow$ il grafo è tutto un unico blocco
 - Presente una sola componente connessa
- **Definizione Grafo Completo**
 - Un albero è definito completo quando per ogni nodo presenta sempre 2 figli
 - Le foglie devono essere tutte allo stesso livello
 - Gli **alberi completi** sono un **sottoinsieme proprio** degli **alberi bilanciati**
 - Ogni albero completo è anche bilanciato (in senso stretto), ma non tutti gli alberi bilanciati sono completi (un albero bilanciato potrebbe non essere completamente pieno a ogni livello).
- **Complessità dell'algoritmo che inverte il grafo**
 - L'inversione di un grafo richiede di visitare tutti gli archi e ricostruirli al contrario
 - $(u,v) \rightarrow (v,u)$
 - Complessità:
 - Liste di adiacenza $\rightarrow O(V + E)$
 - Matrice di adiacenza $\rightarrow O(V^2)$
- **Punto di articolazione + Bridge**
 - Punto di Articolazione $\rightarrow$ vertice la cui rimozione disconnette il grafo
 - Rimuovendo il vertice si rimuovono anche gli archi su esso insistenti
 - Dato l'albero della DFS G' :

- La radice di G' è punto di articolazione $\leftrightarrow$ ha almeno 2 figli
 - Ogni altro vertice v è punto di elaborazione $\leftrightarrow v$ ha un figlio s tale che non vi è alcun arco B da s o un suo discendente ad un antenato proprio di v
- Bridge $\rightarrow$ arco la cui rimozione disconnette il grafo
- Un arco B non può essere un ponte $\rightarrow$ i due vertici connessi da archi T
- Un arco T è ponte $\leftrightarrow$ non esistono archi B che connettono un discendente di w ad un antenato di v nell'albero della DFS
- Algoritmo banale:
 - Rimuovo archi uno alla volta e verificare se grafo rimane connesso
- **IBST: cos'è, chiave di ricerca, a cosa serve il massimo**
 - Un BST che nell'Item presenta una struct con coppia ordinata di reali low ed high
 - Due intervalli i ed i' hanno intersezione $\leftrightarrow$
 - $low[i] \leq high[i'] \ \&\& \ low[i'] \leq high[i]$
 - Chiave di ricerca $\rightarrow$ campo low del nodo
 - Bisogna aggiungere un'informazione aggiuntiva per ogni nodo:
 - Il massimo dei campi max dei sottoalberi e del nodo stesso
 - Fondamentale per individuare overlap
 - Search (ricerca nodo h con intervallo che interseca intervallo i):
 - Percorro albero dalla radice
 - Ricorro da nodo h :
 - sottoSX se $h \rightarrow l \rightarrow max \geq low[i]$
 - sottoDX se $h \rightarrow l \rightarrow max < low[i]$
 - Terminazione:
 - Trovato intervallo che interseca i
 - Oppure trovato albero vuoto
- **Definizione di grafi statici e semistatici:**
 - Statici: non si aggiungono né si cancellano NE' VERTICI NE' ARCHI
 - Semi-Statici: non si aggiungono né si cancellano VERTICI, archi si può
 - Dinamici: si possono aggiungere e cancellare sia vertici sia archi
- **Paradigma greedy (cos'è e come funziona):**
 - E' un'alternativa al divide et impera ed alla programmazione dinamica
 - Di minor Complessità
 - NON SEMPRE RITORNA SOLUZIONE OTTIMA
 - Principi:
 - Ad ogni passo scelgo sol localmente ottima $\rightarrow$ tento di trovare sol globalmente ottima

- NO BACKTRACK → scelte fatte non vengono riconsiderate
 - Funzione di APPETIBILITA' → grazie a cui facciamo scelte localmente ottime
- Vantaggi:
 - Algoritmo molto semplice
 - Tempo di elaborazione molto ridotto
- Svantaggi:
 - Soluzione non necessariamente ottima → non è detto che lo spazio delle possibilità sia esplorato in maniera esaustiva
- Algoritmo:
 - Appetibilità sono note in partenza e non modificate
 - Soluzione inizialmente vuota
 - Ordino le scelte per appetibilità decrescenti
 - Scelte in ordine decrescente → se possibile aggiungo alla soluzione
 - Appetibilità modificabili:
 - Uso una coda a priorità
- LIS
 - Dati N interi → SOTTOSEQUENZA una qualsiasi n-upla di $k \leq N$ elementi con INDICI CRESCENTI
 - Sottosequenza → indici non necessariamente contigui
 - Sottostringa → indici contigui
 - Prefisso → elementi $(X_0, \dots, X_i)$ se di lunghezza $i+1$
 - LIS è la sottosequenza di lunghezza massimale con EL DI VALORI CRESCENTI
 - Complessità
 - Con modello del powerset (combinatorio) → $O(2^N)$
 - Con programmazione dinamica se:
 - Sottostruttura ottima → applicabile
 - Numero polinomiale di sottoproblemi indipendenti → conveniente
 - → $O(N^2)$
 - Applicabilità/Convenienza:
 - Esistenza sottostruttura ottima:
 - Data sol ottima, se non fosse ottima la sol per ogni prefisso (sottoproblema), ne potremmo trovare una migliore → la soluzione non sarebbe ottima → ASSURDO
 - Numero sottoproblemi indipendenti polinomiale $O(N)$

- Soluzione di ogni sottoproblema $\rightarrow O(N)$
- Calcolo bottom-up del valore:
 - $val[] \rightarrow$ per gli N interi
 - $L[] \rightarrow$ per memorizzare la lunghezza della LIS per ogni prefisso i -esimo
 - $P[] \rightarrow$ per memorizzare l'indice del predecessore nella LIS
 - $last \rightarrow$ int per memorizzare indice dell'ultimo elemento nella LIS
- Passi:
 - Min val di una LIS è 1
 - Per lunghezze crescenti del sottovettore:
 - Trovo LIS del prefisso i che soddisfa le condizioni
 - Registro in P indice del valore precedente
- **Codici liberi da prefisso:**
 - Codici in cui nessuna parola di codice valida è un prefisso di un'altra
 - Codifica $\rightarrow$ giustapposizione di stringhe
 - Decodifica $\rightarrow$ percorrimiento di un albero binario
 - Codici a lunghezza fissa:
 - Bit usati $\rightarrow n = \log_2(\text{cardinalità dei simboli})$
 - Facile decodifica
 - SIMBOLI ISOFREQUENZIALI
 - Codici a lunghezza variabile:
 - Difficile decodifica
 - Risparmio di memoria
 - SIMBOLI CON DIVERSE FREQUENZE
 - Come funziona:
 - PQ con frequenze crescenti
 - Inizialmente ogni simbolo = foglia
 - Passo i -esimo:
 - Estraggo 2 simboli a minor frequenza
 - Li aggrego in un BST $\rightarrow$ simbolo come nodo, freq come somma delle freq
 - Inserisco nella PQ
 - Coda vuota $\rightarrow$ terminazione
- **Tabella ad accesso diretto**
 - Nelle tabelle ad accesso diretto la chiave k funge da indice di un array $st[]$
 - Usate per inserzione, ricerca e cancellazione
 - Non per ordinamento e selezione
 - Limiti tabelle accesso diretto:

- U molto grande $\rightarrow$ st[] non allocabile
 - $K \ll U \rightarrow$ spreco di memoria
- ADT con occupazione di spazio $\rightarrow O(K)$
- Funzione di hash $h(k)$ trasforma chiave in un indice della tabella
 - Non può essere perfetta $\rightarrow$ ci saranno sempre collisioni, quindi sono da minimizzare
 - Vogliamo che indici siano equiprobabili
 - Nel caso reale le CHIAVI non lo sono
 - Dobbiamo scorrelare $h(k_i)$ da $h(k_j)$
 - Amplifichiamo differenze
 - Scorrelare $h(k)$ da k
 - Dobbiamo distribuire gli $h(k)$ in modo uniforme
 - Usare tutti i bit della chiave
 - Moltiplicare per un numero primo
- **Linear chaining ed Open Addressing + complessità per ognuna:**
 - **Linear chaining** $\rightarrow$ più elementi nella stessa casella
 - Uso lista concatenata
 - Operazioni:
 - Inserimenti in testa alla lista
 - Ricerca nella lista
 - Cancellazione dalla lista
 - Determinazione della dimensione M della tabella
 - Il più piccolo primo $M \geq (\text{numero di chiavi max})/r$
 - Così lunghezza media liste = r
 - Fattore di carico $\alpha = N/M$
 - Complessità:
 - Inserimento $\rightarrow O(1)$
 - Ricerca:
 - Caso peggiore $\rightarrow O(N)$, unica lista, N elementi
 - Caso medio $\rightarrow O(1 + \alpha)$
 - Cancellazione:
 - Come ricerca se conosco il valore di x, lista linkata semplice
 - $O(1)$ se disponibile puntatore ad x, lista doppiamente linkata
 - **Open Addressing** $\rightarrow$ ogni cella un solo elemento $\rightarrow N \leq M$, $\alpha \leq 1$
 - Se collisione $\rightarrow$ ricerca cella non occupata tramite PROBING
 - Permutazione delle celle $\rightarrow$ ritento con indici diversi

- Linear, Quadratic Probing e Double Hashing
- Usato quando NON SI CANCELLA MAI
 - La cancellazione è un'operazione complessa, interrompe catene di collisione
 - Soluzioni:
 - SOLUZIONE 1: Sostituisco chiave cancellata con chiave sentinella
 - Conta come piena in ricerca
 - Conta come vuota in inserzione
 - SOLUZIONE 2: Reinserisco le chiavi del cluster sottostante la chiave cancellata
- Un problema di Open Addressing è il CLUSTERING
 - Il raggruppamento di posizioni occupate contigue
- **Linear Probing:**
 - Calcolo $i = h(k)$
 - Se libero, inserisco la chiave
 - Altrimenti $i = (i+1) \% M$
 - Ripeto fino a cella vuota
 - La ricerca fa esattamente così, se non trova la chiave cercata
 - Complessità probing per ricerca:
 - Search miss $\rightarrow 1/2 * (1 + 1/(1 - \alpha))$
 - Search hit $\rightarrow 1/2 * (1 + 1/(1 - \alpha)^2)$
- **Quadratic Probing:**
 - i è il contatore dei tentativi $\rightarrow$ inizialmente $i = 0$
 - $index = (h(k) + c1*i + c2*i^2) \% M$
 - Se libero inserisco chiave, altrimenti $i++$ e ripeto fino a cella vuota
 - Se $M = 2^k \rightarrow c1 = c2 = 1/2$
 - Abbiamo collisioni più distribuite nella tabella
- **Double Hashing:**
 - $i = h1(k)$
 - Se libero inserisco, altrimenti calcolo $j = h2(k)$
 - provo in $i = (i + j) \% M$
 - Ripeto fino a cella vuota
 - i non deve mai ripetersi $\rightarrow$ altrimenti ciclo infinito
 - $h1$ mai = 0
 - $h2 \% M$ mai = 0
 - Solitamente:
 - $h1(k) = k \% M$

- $h_2(k) = 1 + k \% 97$
 - Complessità probing per ricerca:
 - Search miss $\rightarrow 1/(1 - \alpha)$
 - Search hit $\rightarrow 1/\alpha * \ln(1/(1 - \alpha))$
- **STdelete nel caso di linear chaining e nel caso di open addressing**
 - Linear chaining:
 - Ogni cella contiene una lista $\rightarrow$ più elementi possibili nella stessa cella
 - Per fare delete:
 - Cerco l'elemento da cancellare nella lista corrispondente e lo rimuovo tramite cambio dei link
 - Non sono presenti problemi nella cancellazione
 - Open addressing:
 - Viene usato quando NON SI CANCELLA MAI
 - La cancellazione è un'operazione complessa, interrompe catene di collisione
 - Soluzioni:
 - SOLUZIONE 1: Sostituisco chiave cancellata con chiave sentinella
 - Conta come piena in ricerca
 - Conta come vuota in inserzione
 - Così facendo durante una ricerca viene ignorata, ma durante inserzione possono utilizzare quella cella
 - SOLUZIONE 2: Reinserisco le chiavi del cluster sottostante la chiave cancellata
- **DFS e complessità**
 - Dato un grafo (connesso o non connesso), parto da un vertice $s \rightarrow$ visito tutti i vertici del grafo, raggiungibili da s e non
 - Principio base della DFS
 - Espando ultimo vertice scoperto che ha ancora vertici non ancora scoperti adiacenti
 - Scoperta di un vertice:
 - Prima volta che si incontra nella visita (discesa ricorsiva, pre-order)
 - $pre[]$ come vettore mark $\rightarrow$ contiene tempi crescenti di scoperta
 - Algoritmo:

- GRAPHdfs → a partire da vertice, visita tutti i vertici del grafo, chiamata ricorsiva dfsR → termina quando tutti i vertici sono stati visitati
 - dfsR → visita in profondità partendo da vertice id identificato con un arco fittizio EDGEcreate(id,id) → utile in fase di visualizzazione
 - Termina quando ha visitato in profondità tutti i nodi raggiungibili da id
- Strutture dati:
 - Grafo non pesato come lista di adiacenze
 - pre[] → per ogni vertice registra tempo di scoperta
 - post[] → tempo fine elaborazione vertice, uscita ricorsione, visita post-order
 - st[] → identifica padre nella DFS
 - cnt → contatore per tempi di scoperta
- Archi etichettati come T,B,F,C
- Complessità:
 - Lista adiacenze → $O(V + E)$
 - Inizializzazione → $O(V)$
 - Visita ricorsiva da u → $O(E)$
 - Matrice adiacenza → $O(V^2)$
- **BFS e complessità**
 - Partendo da un vertice:
 - Visito tutti i vertici raggiungibili da s
 - Non necessariamente tutti i vertici come DFS
 - Calcola distanza minima da s di tutti i vertici da esso raggiungibili
 - Genera un albero della visita in ampiezza
 - Ampiezza:
 - Espande tutta la frontiera tra vertici già scoperti / non ancora scoperti
 - Strutture dati:
 - Grafo non pesato come matrice di adiacenze
 - Coda Q → di archi e = (v,w)
 - st[] → registro il padre nell'albero della BFS
 - pre[] → registro tempo di scoperta dei vertici
 - time → contatore del tempo
 - Algoritmo:
 - Metto arco fittizio e = (id,id) nella coda
 - Ripeto fino a quando coda non si svuota:

- Estraggo dalla coda un arco $e = (v, w)$
- Se $e.w$ vertice NON ANCORA SCOPERTO ($pre[w] == -1$):
 - $st[e.w] = e.v$
 - $pre[e.w] = time++$
 - Trovo i vertici non ancora scoperti adiacenti a $e.w$
 - Metto nella coda archi che li collegano a $e.w$
- Complessità:
 - Scansione matrice delle adiacenze $\rightarrow O(V^2)$
 - Con lista delle adiacenze $\rightarrow O(V + E)$
- Funzioni tail recursive cosa sono + possibile convertirle in iterative? + non tail recursive si possono convertire? + perché non generano stack overflow
 - Una funzione è detta tail-recursive se CHIAMATA RICORSIVA ULTIMA OPERAZIONE DA ESEGUIRE
 - Ovviamente senza considerare il return

```
unsigned long tr_fact(int n, unsigned long f)
{
    if(n == 0)
        return f;
    return tr_fact(n-1, n*f);
}
```

FUNZIONE CHE FA MOLTIPLICAZIONE

- In questo caso la moltiplicazione viene eseguita PRIMA della chiamata ricorsiva $\rightarrow$ CALCOLI IN FASE DI DISCESA

```
unsigned long fact(int n) {
    if(n == 0)
        return 1;
    return n*fact(n-1);
}
```

- Qui la moltiplicazione eseguita DOPO il ritorno della chiamata ricorsiva $\rightarrow$ CALCOLI IN FASE DI RISALITA
 - NON TAIL-RECURSIVE
- Vantaggi Tail-Recursive:
 - La funzione chiamante deve solo ritornare il valore calcolato dalla funzione chiamata $\rightarrow$ non ha ulteriori calcoli da eseguire
 - La pop dello stack frame della funzione chiamante avviene PRIMA della push nello stack frame della funzione chiamata
 - Lo stack frame della funzione chiamata quindi RIMPIAZZA semplicemente quello della funzione chiamante
 - L'occupazione di memoria è SEMPRE $O(1)$
 - Non più $O(n)$, se n livelli di ricorsione

- NON CI PUO' ESSERE STACK OVERFLOW
- Svantaggi Tail-Recursive:
 - Funzione più difficile da scrivere
 - Non tutti i compilatori (come quello del C) riescono a sfruttare queste funzioni per ottimizzare il codice
- Da ricorsiva ad iterativa:
 - O iterativa “nativa” o ricorsiva
 - Soluzioni possibili per ricorsiva:
 - Se funz tail-recursive → trasformabile direttamente in iterativa senza uso di stack
 - Utilizzo dei cicli (es. while)
 - Se funz NON tail-recursive → trasformabile in iterativa CON GESTIONE DELLO STACK A CARICO DELL'UTENTE
 - Meno efficienti delle iterative “native”
- **Cammino di Hamilton**
 - Dato un grafo non orientato e 2 suoi vertici v e w
 - Cammino di Hamilton → cammino semplice che li connette VISITANDO OGNI VERTICE UNA ED UNA SOLA VOLTA
 - Se v coincide con w → CICLO DI HAMILTON
 - Algoritmo:
 - Qualunque vertice t adiacente a v → determino ricorsivamente se esiste cammino semplice da t a w
 - Ritorno successo $\leftrightarrow$ lunghezza del cammino = $V - 1$
 - visited[] per marcare nodi già visitati
 - faccio reset cella se backtrack → ritornato insuccesso
 - Complessità esponenziale
- **Cosa è un cammino di Eulero e la condizione per far sì che esso esista**
 - Dato un grafo non orientato e 2 suoi vertici v e w
 - Cammino di Eulero → cammino ANCHE NON SEMPLICE che li connette ATTRAVERSANDO OGNI ARCO UNA ED UNA SOLA VOLTA
 - Se v coincide con w → CICLO DI EULERO
 - Condizione di esistenza:
 - Grado di un vertice → numero di archi incidenti a quel vertice
 - Quanti archi entrano/escono
 - G non orientato ha CAMMINO di eulero $\leftrightarrow$ è connesso ed esattamente 2 vertici di grado DISPARI
 - G non orientato ha CICLO di eulero $\leftrightarrow$ è connesso e TUTTI i suoi vertici sono di grado pari

- Selezione di attività, quali algoritmi e approcci si possono usare per risolvere il problema

1. PROBLEMA 1:

- Insieme di n attività con tempi di inizio (s) e tempo di fine (f)
- Output → insieme con il MASSIMO NUM DI ATTIVITA' COMPATIBILI
- Compatibilità = due intervalli non si sovrappongono:
 - $s_i \geq f_j$ oppure $s_j \geq f_i$
- APPROCCIO GREEDY → ordino attività in base a funzione di appetibilità
- Funzioni di appetibilità che NON PORTANO SEMPRE A SOL OTTIME:
 - Ordinamento per tempo di inizio crescente:
 - Att che inizia presto ma dura a lungo impedisce di selezionarne altre
 - Ordinamento per durata crescente:
 - Att breve che interseca 2 lunghe impedisce di selezionarle
- Funzione che PORTA SEMPRE A SOL OTTIME:
 - Ordinamento per fine di tempo crescente:
 - Se k è l'attività che finisce per prima:
 - Scegliere k non pregiudica nulla circa le attività che iniziano dopo la sua fine
 - Se ci sono attività che iniziano prima di k , visto che finiscono dopo k , si intersecano e quindi se ne sarebbe potuta scegliere solo una → si sceglie proprio k

2. PROBLEMA 2:

- Insieme di n attività con tempi di inizio (s) e tempo di fine (f)
- Output → insieme con SOMMA DELLE DURATE MASSIMA
- Ordinare per tempo di fine crescente → NO SOLUZIONE OTTIMA
- Per i cammini massimi si può applicare la programmazione dinamica?
 - Vogliamo trovare un cammino semplice formato dal massimo numero di archi
 - Se non fosse semplice → percorrendo il ciclo infinite volte ricorrerei infinite volte
 - Il problema perderebbe di significato
 - Il problema però non presenta una sottostruttura ottima
 - Troviamo sempre un sottocammino maggiore rispetto a quello considerato
 - Combinando i sottocammini semplici e massimi → cammino ottenuto non è semplice
 - PROGRAMMAZIONE DINAMICA NON APPLICABILE

- Problema dei cammini massimi è NP-completo
 - NP = “ Nondeterministic Polynomial time ”
- La programmazione dinamica viene utilizzata nel calcolo dei cammini minimi? Come dimostri che è applicabile?
 - Dato un grafo orientato e non pesato e 2 suoi vertici u e v:
 - Trovare il cammino minimo, ovvero formato dal minimo num di archi
 - E' necessariamente semplice → se non lo fosse potrei eliminare il ciclo e ridurre il numero di archi
 - Cammino minimo tra u e v nel grafo $G = (V, E)$
 - Problema banale se u e v coincidono
 - Esiste un cammino minimo $p: u \rightarrow (p) v$
 - Essendo u diverso da v, esiste un nodo intermedio w che spezza il cammino p in $p_1: u \rightarrow (p_1) w$ e $p_2: w \rightarrow (p_2) v$
 - Se p_1 (o p_2) non fosse ottimo, esisterebbe un p_1' ottimo, che sostituito a p_1 porterebbe a dire che p non è ottimo
 - ASSURDO
 - Quindi il problema ha sottostruttura ottima
 - PROGRAMMAZIONE DINAMICA APPLICABILE
- StackFrame + Stack di sistema
 - Stack:
 - Tipo di dato astratto ADT, supporta operazioni di :
 - Push → inserimento oggetto in cima allo stack
 - Pop → prelievo e cancellazione dalla cima dello stack
 - Strategia di gestione dei dati → LIFO
 - Stack frame:
 - Struttura dati che contiene almeno:
 - Parametri formali
 - Variabili locali
 - Indirizzo a cui si ritorna al termine della funzione
 - Puntatore a codice della funzione
 - Contiene quello che serve per la chiamata di una funzione
 - Viene creato alla chiamata della funzione e distrutto al suo termine
 - Push nello stack alla chiamata → $\text{push}(f_i)$
 - Pop dallo stack al termine → $\text{pop}()$, non ha argomenti, fa pop sempre dalla cima dello stack
 - Stack frame memorizzati nello stack di sistema
 - Stack di sistema ha però una dimensione prefissata di memoria

- Stack cresce da indirizzi maggiori a minori
- Stack pointer SP:
 - Registro che contiene indirizzo del primo stack frame disponibile
- Se troppe funzioni ricorsive
 - Stack di sistema si riempie → STACK OVERFLOW
- Karatsuba + complessità
 - Prodotto di 2 interi x ed y:
 - Algoritmo elementare → moltiplicazione di ogni cifra del moltiplicatore per le cifre del moltiplicando, scalamento a SX (moltiplicazione per 10) e somme
 - Richiede conoscenza tabelline pitagoriche
 - Algoritmo ricorsivo elementare:
 - Se $y = 1 \rightarrow x * 1 = x$
 - Altrimenti $\rightarrow x * y = x + x * (y - 1)$
 - Algoritmo ricorsivo per moltiplicazione di 2 interi positivi x ed y di n cifre ($n = 2^k$)
 - Se dimensione $n = 1 \rightarrow$ calcolo $x*y$ (tabelline pitagoriche)
 - Se $n > 1$:

Proviamo a
spezzare il num. di cifre

divide and conquer
 $a = 4 \quad b = 2$

Algoritmo ricorsivo per la moltiplicazione di 2 interi positivi x e y di n cifre (con $n = 2^k$):

- se la dimensione è $n=1$ calcola $x * y$ (tabelline pitagoriche)
- per $n > 1$
 - dividi x in 2: $x = 10^{n/2} * x_s + x_d$
 - dividi y in 2: $y = 10^{n/2} * y_s + y_d$
 - calcola ricorsivamente $x_s * y_s, x_s * y_d, x_d * y_s, x_d * y_d$
 - calcola $x * y = 10^n * x_s * y_s + 10^{n/2} * (x_s * y_d + x_d * y_s) + x_d * y_d$

▪ Complessità:

- Shift a SX (moltiplicazione per 10) → ognuna $O(1)$ → totale $O(k)$
 - Al più $k = n$, necessarie 2 moltiplicazioni → $O(n)$
- Somma di numeri su k cifre → $O(k)$
 - Due num di n cifre, prodotto su 2n cifre
 - Al più $k = 2n$, necessarie 3 somme → $O(n)$

- Moltiplicazione → costo della ricorsione (4 moltiplicazioni)
- TOTALE COMPLESSITA' → $O(n^2)$

○ Karatsuba:

- Si cerca di ottimizzare l'algoritmo di base → riduce numero di moltiplicazioni → 3 MOLTIPLICAZIONI RICORSIVE (non 4)

$$x_s * y_d + x_d * y_s = x_s * y_s + x_d * y_d - (x_s - x_d) * (y_s - y_d)$$

- Prima divide and conquer con $a = 4$ e $b = 2$ → ora $a = 3$
- Complessità:
 - $D(n)$, divisione → $O(1)$
 - $C(n)$, ricombinazione → $O(n)$
 - Come prima ma ora $a = 3$
 - TOTALE COMPLESSITA' → $O(n^{\log_2(3)})$

• Powerset: cosa sono e algoritmi per trovarli tutti con descrizione

- Dato un insieme S di k elementi → powerset è l'insieme dei sottoinsiemi di S , incluso S stesso e l'insieme vuoto
- Si può calcolare ricorrendo a 3 modelli
 - Paradigma divide et impera
 - Disposizioni ripetute
 - Combinazioni semplici
- Divide et impera:
 - Caso Terminale → insieme il cui unico elemento è l'insieme vuoto
 - Caso Ricorsivo → insieme formato dall'unione :
 - Insieme delle parti per $n-1$ elementi
 - Qualunque i con gli insiemi che risultano dall'unione di ognuno degli insiemi delle parti di i (p_i) che appartengono all'insieme delle parti per $n-1$ elementi con l'insieme che contiene l'elemento i -esimo S_i

$$P(S_n) = \begin{cases} \{\emptyset\} & \text{se } n = 0 \\ P(S_{n-1}) \cup \{p_i \cup \{s_i\} \mid p_i \in P(S_{n-1})\} & \text{se } n > 0 \end{cases}$$

- Si usano 2 rami ricorsivi distinti, a seconda che elemento corrente sia incluso o meno nella soluzione
- In sol si memorizza direttamente l'elemento
- Indice start per escludere soluzioni simmetriche
- Viene ritornato cnt → numero totale di insiemi

```

int powerset(int pos,int *val,int *sol,int n,int start,int cnt) {
    int i;
    if (start >= n) {
        for (i = 0; i < pos; i++)
            printf("%d ", sol[i]);
        printf("\n");
        return cnt+1;
    }
    for (i = start; i < n; i++) {
        sol[pos] = val[i];
        cnt = powerset(pos+1, val, sol, n, i+1, cnt);
    }
    cnt = powerset(pos, val, sol, n, n, cnt);
    return cnt;
}

```

terminazione: non ci sono più elementi

per tutti gli elementi da start in poi

includi elemento e ricorri

non aggiungere nulla e ricorri

○ Disposizioni Ripetute:

- Ogni sottoinsieme rappresentato dal vettore sol[] di n elementi
 - sol[pos] = 0 → oggetto pos-esimo NON appartiene all'insieme
 - sol[pos] = 1 → oggetto pos-esimo appartiene all'insieme
 - Nella stessa soluzione 0 e 1 possono comparire più volte

```

int powerset(int pos,int *val,int *sol,int n,int cnt) {
    int j;
    if (pos >= n) {
        printf("{ \t");
        for (j=0; j<n; j++)
            if (sol[j]!=0)
                printf("%d \t", val[j]);
        printf("} \n");
        return cnt+1;
    }
    sol[pos] = 0;
    cnt = powerset(pos+1, val, sol, n, cnt);
    sol[pos] = 1;
    cnt = powerset(pos+1, val, sol, n, cnt);
    return cnt;
}

```

terminazione: stampa soluzione

non prendere l'elemento pos

ricorri su pos+1

backtrack: prendi l'elemento pos

solo se è messo (sol[j] != 0)

○ Combinazioni semplici:

- Unione di insieme vuoto (non viene contato altrimenti) ed insieme delle parti con $j = 1, 2, 3, \dots, n$ elementi
- Ordine non conta
- Combinazioni semplici di n elementi presi a gruppi di j
- Il wrapper si occupa dell'unione dell'insieme vuoto e di iterare la chiamata alla funzione ricorsiva delle combinazioni

```

int powerset(int *val, int n, int *sol){
    int cnt = 0, j;
    printf("{ }\\n");
    cnt++;
    for(j = 1; j <= n; j++){
        cnt += powerset_r(val, n, sol, j, 0, 0);
    }
    return cnt;
}

```

insieme vuoto

iterazione delle chiamate ricorsive

```

int powerset_r(int* val, int n, int *sol, int j, int pos, int start){
    int cnt = 0, i;
    if (pos >= j){
        printf("{ ");
        for (i = 0; i < j; i++){
            printf("%d ", sol[i]);
        }
        printf("}\\n");
        return 1;
    }
    for (i = start; i < n; i++){
        sol[pos] = val[i];
        cnt += powerset_r(val, n, sol, j, pos+1, i+1);
    }
    return cnt;
}

```

caso terminale: raggiunto numero prefissato di elementi

per tutti gli elementi da start in poi